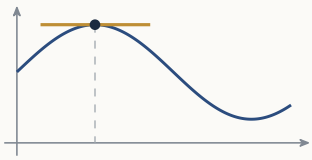


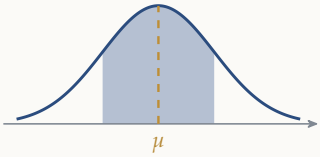
$Ax = b$

ALGÈBRE LINÉAIRE



$f'(x) = 0$

ANALYSE



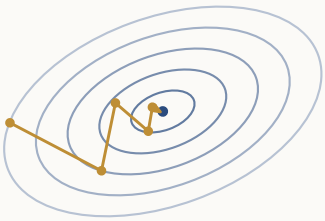
$\mathcal{N}(\mu, \sigma^2)$

PROBABILITÉS

Machine Learning For Engineer

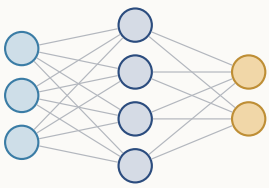
Les mathématiques de l'intelligence artificielle, reprises depuis zéro

Aucun prérequis : on part vraiment de zéro.



$\theta \leftarrow \theta - \eta \nabla \mathcal{J}(\theta)$

OPTIMISATION



$y = \sigma(Wx + b)$

APPRENTISSAGE

Doctor Mokira

2026

Édition 2026. Document composé avec LaTeX (pdfTeX), polices Palatino (texte et mathématiques), TeX Gyre Heros (titres) et DejaVu Sans Mono (code). Figures et schémas réalisés avec TikZ et pgfplots, coloration syntaxique assurée par minted/Pygments, encadrés par tcolorbox, bibliographie par biblatex.

Ce livre rassemble en un seul volume les mathématiques dont l'intelligence artificielle et la science des données ont réellement besoin : algèbre linéaire, analyse, probabilités, statistique et optimisation. Il est écrit pour être lu sans aucun prérequis : on part de zéro, on explique avec des mots avant d'écrire des symboles, et on n'avance jamais sans avoir d'abord dit pourquoi.

Avant-propos

« Un livre de mathématiques ne devrait effrayer personne. »

L'intelligence artificielle repose entièrement sur des mathématiques. Le problème, c'est que les livres qui les enseignent supposent presque tous que vous les connaissez déjà : ils vont vite, ils écrivent en symboles avant d'avoir expliqué en mots, et ils supposent une aisance en programmation que peu de lecteurs ont au départ. Résultat : beaucoup de gens curieux, capables, motivés, s'arrêtent à la première page.

Ici, on ne suppose presque rien. Chaque notion est d'abord racontée avec des mots simples, puis illustrée par une image ou un dessin, puis seulement écrite en langage mathématique — et enfin mise en pratique avec quelques lignes de code que vous pouvez exécuter vous-même.

> **En clair**
>
> Notre promesse : si vous savez additionner, multiplier et lire un tableau de nombres, vous pouvez comprendre les mathématiques de l'intelligence artificielle. Le reste, on le construit ensemble, dans l'ordre, sans sauter d'étape.

Pour qui est ce livre

Pour toute personne qui veut appliquer les mathématiques ailleurs — informatique, IA, traitement du signal, analyse de données — et qui part d'un niveau nul, ancien ou fragile. Aucun diplôme n'est demandé. Le texte est écrit pour rester lisible par un élève de début de collège lorsque le sujet le permet, et pour ne jamais perdre le lecteur lorsque le sujet devient réellement difficile.

C'est aussi un *tout-en-un* : plutôt que d'aller chercher l'algèbre linéaire dans un livre, les probabilités dans un deuxième et l'optimisation dans un troi-

sième — chacun avec ses notations et ses habitudes —, vous trouverez ici l'ensemble, avec des notations cohérentes d'un bout à l'autre.

Comment lire ce livre

Le texte utilise quelques encadrés récurrents, faciles à reconnaître à leur pictogramme :

- **En clair** — l'idée expliquée avec des mots de tous les jours.
- **L'intuition** — l'image mentale à garder en tête.
- **À quoi ça sert en IA ?** — où cette notion apparaît vraiment dans un algorithme d'apprentissage.
- **Sur des données réelles** — la même idée, appliquée à un jeu de données concret.
- **Ce qu'il faut savoir avant** — les seuls prérequis de la section, et où les réviser.
- **Piège à éviter** — l'erreur classique que tout le monde fait.
- **À retenir** — l'essentiel, si vous ne deviez garder qu'une chose.
- **Pour aller plus loin** — un pont vers des sujets plus avancés, que vous pouvez ignorer en première lecture.

Les résultats importants sont énoncés comme *théorèmes*, et le code qui les met en œuvre (surtout en Python) est fourni et commenté. Les démonstrations sont là pour ceux qui veulent savoir *pourquoi* : sautez-les sans remords à la première lecture, le fil du texte reste compréhensible sans elles.

Bonne lecture.

Table des matières

Avant-propos iii

Table des matières v

I Le langage des données : algèbre linéaire 1

1 Vecteurs, matrices et normes 3

1.1 Un vecteur, c'est juste une liste de nombres 3

1.2 Une matrice transforme les vecteurs 6

1.3 Passons au code 7

Exercices 8

II Le langage de l'incertain : probabilités et statistique 11

III Bouger dans la bonne direction : calcul différentiel et optimisation 13

IV Apprendre à partir de données : apprentissage automatique 15

V Les réseaux de neurones, démontés pièce par pièce 17

Bibliographie 19

Première partie

Le langage des données : algèbre linéaire

CHAPITRE 1

Vecteurs, matrices et normes

Avant de faire apprendre quoi que ce soit à une machine, apprivoisons les objets avec lesquels elle travaille : les vecteurs et les matrices. Ce chapitre pose le décor — en douceur.

Ce qu'il faut savoir avant

Savoir additionner et multiplier des nombres, et savoir ce qu'est une racine carrée. Rien d'autre. Si le symbole $\sum$ vous inquiète, ne vous arrêtez pas : il est expliqué la première fois qu'il apparaît.

1.1 Un vecteur, c'est juste une liste de nombres

En clair

Un *vecteur* de taille n, c'est une liste ordonnée de n nombres. Rien de plus. Par exemple, la position d'un point sur une carte (x, y) est un vecteur de taille 2. Les pixels d'une image, mis bout à bout, forment un vecteur (très long).

À quoi ça sert en IA ?

En intelligence artificielle, **tout** devient un vecteur : une photo, une phrase, un client, une molécule. C'est la première chose que fait n'im-

porte quel système d'IA — transformer le monde en listes de nombres, parce que c'est la seule chose qu'une machine sache calculer. Le vecteur qui représente un objet s'appelle son *plongement* (*embedding* en anglais).

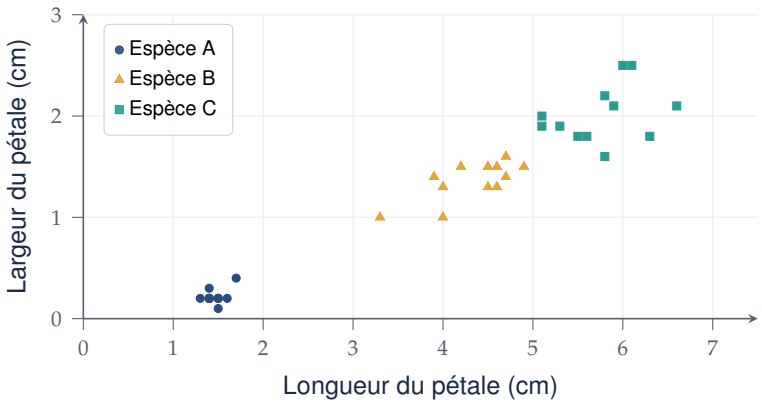


FIGURE 1.1. Un jeu de données, c'est un nuage de vecteurs. Ici, chaque fleur est décrite par deux nombres seulement — donc par un vecteur de $\mathbb{R}^2$ —, et l'espèce se lit presque à l'œil nu.

On note un vecteur en gras, $\boldsymbol{x} \in \mathbb{R}^n$, et ses composantes $x_1, x_2, \ldots, x_n$. On les empile verticalement :

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}.$$

L'intuition

Voyez le vecteur $\boldsymbol{x} \in \mathbb{R}^2$ comme une *flèche* partant de l'origine et pointant vers le point (x_1, x_2). Additionner deux vecteurs, c'est mettre les flèches bout à bout.

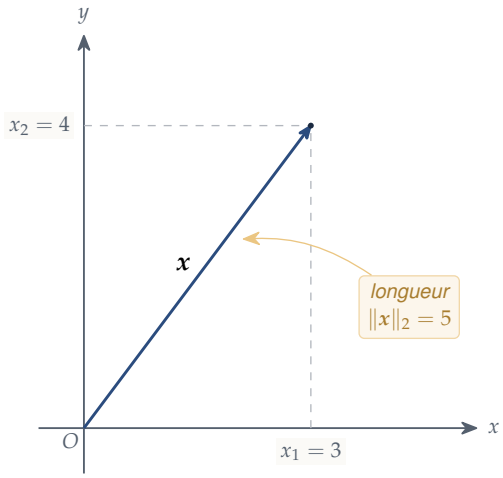


FIGURE 1.2. Le vecteur $\boldsymbol{x} = (3, 4)$ vu comme une flèche. Sa longueur se lit directement avec le théorème de Pythagore : les pointillés gris forment un triangle rectangle de côtés 3 et 4, dont l'hypoténuse — la flèche — mesure 5.

Définition 1.1 (Norme euclidienne)

La *longueur* d'un vecteur $\boldsymbol{x} \in \mathbb{R}^n$, appelée *norme euclidienne*, est le nombre

$$\|\boldsymbol{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\sum_{i=1}^{n} x_i^2}.$$

C'est exactement le théorème de Pythagore, généralisé à n dimensions.

Exemple 1.2 (Calcul d'une norme)

Pour $\boldsymbol{x} = (3, 4)$, on a $\|\boldsymbol{x}\|_2 = \sqrt{9 + 16} = \sqrt{25} = 5$. La flèche mesure 5 unités de long.

Piège à éviter

La norme n'est **pas** la somme des composantes ! On élève au carré, on additionne, *puis* on prend la racine. Oublier la racine est l'erreur numéro un des débutants.

1.2 Une matrice transforme les vecteurs

Une matrice $A \in \mathcal{M}_{m,n}$ est un tableau de nombres à m lignes et n colonnes. Son vrai rôle : **transformer** un vecteur en un autre, par le produit $\boldsymbol{y} = A\boldsymbol{x}$.

À retenir

Multiplier par une matrice, c'est appliquer une transformation géométrique : une rotation, un étirement, une projection… Comprendre *quelle* transformation cache une matrice, c'est comprendre l'algèbre linéaire.

À quoi ça sert en IA ?

Une couche de réseau de neurones n'est rien d'autre que cela : une multiplication par une matrice, $\boldsymbol{y} = W\boldsymbol{x} + \boldsymbol{b}$, suivie d'une petite fonction appliquée nombre par nombre. Les coefficients de W sont précisément ce que le réseau *apprend*. Entraîner un modèle, c'est chercher les bons nombres à mettre dans des matrices — et rien de plus mystérieux que cela.

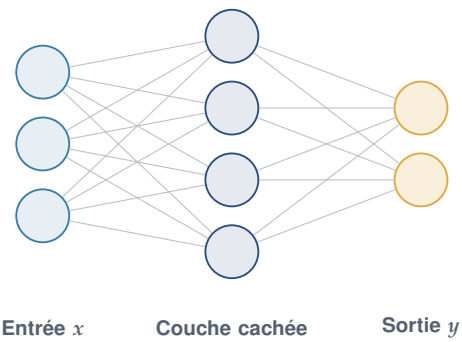


FIGURE 1.3. Un petit réseau de neurones. Chaque trait porte un coefficient : ce sont exactement les cases de la matrice W. Un réseau qui « apprend », c'est un réseau qui ajuste ces nombres.

Théorème 1.3 (Existence et unicité de la solution)

Soit $A \in \mathcal{M}_n$ une matrice carrée. Le système linéaire $A\boldsymbol{x} = \boldsymbol{b}$ admet une *unique* solution pour tout $\boldsymbol{b} \in \mathbb{R}^n$ si et seulement si A est *inversible*, c'est-à-dire $\det(A) \neq 0$. Dans ce cas, $\boldsymbol{x} = A^{-1}\,\boldsymbol{b}$.

Démonstration. Si A est inversible, poser $\boldsymbol{x} = A^{-1}\boldsymbol{b}$ donne $A\boldsymbol{x} = AA^{-1}\boldsymbol{b} = \mathbf{I}\boldsymbol{b} = \boldsymbol{b}$, d'où l'existence ; et si $A\boldsymbol{x} = A\boldsymbol{x}'$ alors, en multipliant à gauche par A^{-1}, on obtient $\boldsymbol{x} = \boldsymbol{x}'$, d'où l'unicité. Réciproquement, si A n'est pas inversible, son noyau $\operatorname{Ker} A$ contient un vecteur non nul et la solution, quand elle existe, n'est jamais unique. ■

Remarque 1.4

En pratique, on ne calcule *presque jamais* A^{-1} pour résoudre $A\boldsymbol{x} = \boldsymbol{b}$: c'est coûteux et numériquement instable. On préfère des méthodes directes (l'élimination de Gauss, au chapitre suivant) ou itératives. Voir aussi GOLUB et VAN LOAN [1].

1.3 Passons au code

Voici comment manipuler tout cela en Python avec `numpy`, la bibliothèque de référence en calcul scientifique [2].

PYTHON • Norme et résolution d'un système linéaire

```python
import numpy as np

# Un vecteur et sa norme euclidienne
x = np.array([3.0, 4.0])
print(np.linalg.norm(x))          # -> 5.0

# Résoudre A x = b SANS inverser A (recommandé)
A = np.array([[2.0, 1.0],
              [1.0, 3.0]])
b = np.array([1.0, 2.0])

x = np.linalg.solve(A, b)         # méthode directe, stable
```

```
print("solution :", x)
print("residu   :", np.linalg.norm(A @ x - b))
```

Le même calcul en Julia, très prisé en recherche numérique :

JULIA • La même résolution en Julia

```julia
using LinearAlgebra

A = [2.0 1.0; 1.0 3.0]
b = [1.0, 2.0]

x = A \ b            # l'opérateur « backslash » résout le système
println("solution : ", x)
println("norme    : ", norm(x))
```

Pour aller plus loin

Le nombre $\mathrm{cond}(A) = \|A\|\,\|A^{-1}\|$, appelé *conditionnement*, mesure à quel point la solution est sensible aux erreurs d'arrondi. Un grand conditionnement annonce des ennuis numériques. Nous y reviendrons en détail.

Exercices

Exercice 1 — Norme et distance

Calculez $\|\boldsymbol{x}\|_2$ pour $\boldsymbol{x} = (1, 2, 2)$, puis la distance entre $\boldsymbol{x}$ et $\boldsymbol{y} = (1, 0, 0)$.

Solution

$\|\boldsymbol{x}\|_2 = \sqrt{1+4+4} = 3$. La distance vaut $\|\boldsymbol{x} - \boldsymbol{y}\|_2 = \|(0, 2, 2)\|_2 = \sqrt{8} = 2\sqrt{2}$.

Exercice 2 — Inversibilité

La matrice $A = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$ est-elle inversible ? Justifiez, puis dites ce que cela implique pour le système $A\boldsymbol{x} = \boldsymbol{b}$.

Solution

$\det(A) = 1 \cdot 4 - 2 \cdot 2 = 0$: la matrice **n'est pas** inversible (la 2e ligne est le double de la 1re). D'après le théorème 1.3, le système n'a pas de solution unique : soit aucune solution, soit une infinité.

Deuxième partie

Le langage de l'incertain : probabilités et statistique

Troisième partie

Bouger dans la bonne direction : calcul différentiel et optimisation

Quatrième partie

Apprendre à partir de données : apprentissage automatique

Cinquième partie

Les réseaux de neurones, démontés pièce par pièce

Bibliographie

[1] G. H. GOLUB et C. F. VAN LOAN. *Matrix Computations*. 4e éd. Baltimore : Johns Hopkins University Press, 2013. ISBN : 9781421407944.

[2] C. R. HARRIS, K. J. MILLMAN, S. J. van der WALT et al. « Array Programming with NumPy ». In : *Nature* 585 (2020), p. 357-362. DOI : 10.1038/s41586-020-2649-2.